Software Testing, Quality Assurance and Maintenance	Winter 2026

Lecture 17 — March 9, 2026

Patrick Lam *version 1*

Property testing with Bolero, plus extension to Kani

We mentioned last time that property testing and bounded model checking looked pretty similar. Let's explore that in more depth. First, we'll talk about the Bolero meta-framework for property testing[1], which can integrate a number of fuzzing engines and Kani as well.

The first example in the blog post isn't executable, but I made an executable version in the `code/L17/use-bolero` subdirectory. We start by talking about property testing, as we've seen with Hypothesis in the Python space and proptest for Rust.

Say we have this function:

```
fn update_account_balance(current_balance: i32, amount: i32) -> i32 {
    return current_balance + amount;
}
```

We can write a regular concrete test like this:

```
fn balance_update_is_correct(current_balance: i32, amount: i32,
    new_balance: i32) -> bool {
 // also: checks that the transaction was carried out correctly,
 // may include other properties as well, e.g. that no overflow occurred,
 // there was enough balance for a withdrawal, etc.
 return current_balance + amount == new_balance;
}

#[test]
fn test_update_account_balance() {
 let current_balance = 5;
 let amount = 2;
 let new_balance = update_account_balance(current_balance, amount);
 assert!(balance_update_is_correct(current_balance, amount, new_balance));
}
```

We can just `cargo test` to run this, and it succeeds of course.

We can implement random testing by hand:

```
#[test]
fn test_update_account_balance_random() {
    let current_balance = rand::random();
    let amount = rand::random();
    let result = update_account_balance(current_balance, amount);
```

[1] https://model-checking.github.io/kani-verifier-blog/2022/10/27/using-kani-with-the-bolero-property-testing-fra
html

```
    assert!(balance_update_is_correct(current_balance, amount, result));
}
```

which is fine, I guess. But we can use coverage-guided fuzzing, as we've discussed. The code in the blog post for using libFuzzer or AFL doesn't actually work, because these fuzzers give you bits, and you have to convert them into i32s yourself. The point, though, is: well, it's pretty much the same interface for random testing, fuzzing, and Kani. So, can't we introduce a layer of abstraction? Bolero does this, and this code does work:

```rust
#[test]
#[cfg_attr(kani, kani::proof)]
fn test_update_account_balance_bolero() {
    bolero::check!().with_type::<(i32, i32)>().cloned().for_each(|(current_balance, amount)| {
        let result = update_account_balance(current_balance, amount);
        assert!(balance_update_is_correct(current_balance, amount, result));
    });
}
```

If we run

```
$ cargo bolero test test_update_account_balance_bolero --engine libfuzzer
```

then it tells us that our code fails on overflow.

```
======================== Test Failure ========================

Input:
(
    369098752,
    1778384896,
)

Error:
panicked at src/main.rs:6:12:
attempt to add with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace.
```

I can't use AFL on my 64-bit computer without doing more admin than I'd like, but presumably it's possible too. There is a honggfuzz backend but my computer doesn't have it. I can use Kani with Bolero, which also shows a failure.

```
SUMMARY:
 ** 1 of 238 failed (1 unreachable)

 ** 1 of 1 cover properties satisfied
```

```
Failed Checks: attempt to add with overflow
 File: "src/main.rs", line 6, in update_account_balance

VERIFICATION:- FAILED
Verification Time: 0.16379055s
```

Running `cargo test` is also supposed to do normal random testing.

Another out-of-bounds example. Maybe we should have just started with this example! Anyway, here it is. It returns the nth-from-the-end element of the input array.

```rust
fn nth_rev(arr: &[i32], index: usize) -> Option<i32> {
    if index < arr.len() {
        let rev_index = arr.len() - index /* - 1 */;
        return Some(arr[rev_index]);
    }
    None
}
```

and we can write a Bolero harness:

```rust
const N: usize = 5;

#[test]
#[cfg_attr(kani, kani::proof)]
fn check_rev() {
    bolero::check!()
        .with_type::<([i32; N], usize)>()
        .cloned()
        .for_each(|(arr, index)| {
            let x = nth_rev(&arr, index);
            if index < arr.len() {
                assert!(x.is_some());
            }
        });
}
```

The `cfg_attr` line marks this as a proof harness only under Kani; if using it normally with Bolero, it's just a test.

We are in bounded model checking land, so there is a limit to the array (5). The test/proof harness creates an array of length 5 and checks that calling `nth_rev` on each array element works. We can use this as a test for Bolero:

```
$ cargo bolero test check_rev --engine libfuzzer
[...]
test failed; shrinking input...

======================== Test Failure ========================

Input:
```

```
(
    [
        0,
        0,
        0,
        0,
        0,
    ],
    0,
)

Error:
panicked at src/main.rs:8:21:
index out of bounds: the len is 5 but the index is 5
```

which says that the index is higher than it should be (the array has elements from 0 to 4). Kani also finds the bug.

```
$ cargo bolero test check_rev --engine kani
[...]
SUMMARY:
 ** 1 of 698 failed (8 unreachable)

 ** 1 of 1 cover properties satisfied

Failed Checks: index out of bounds: the length is less than or equal to the given index
 File: "src/main.rs", line 8, in nth_rev

VERIFICATION:- FAILED
Verification Time: 1.0711443s
```

and if we fix the program, both the fuzzer and Kani succeed. The fuzzer takes a long time.

```
SUMMARY:
 ** 0 of 699 failed (8 unreachable)

 ** 1 of 1 cover properties satisfied


VERIFICATION:- SUCCESSFUL
Verification Time: 1.0993001s
```

Why both? The blog post talks about it. Here's my take. Our example here showed overlap: they succeeded and failed on the same programs. But that's not true in general. Fuzzing might not find an issue in a case where Kani does. Or, for instance, due to bounds, Kani might not find an issue, where fuzzing does.

When either fuzzing or Kani give you a test case, then that's real, and you can fix it. When Kani reports an unwinding error, then you don't know. If Kani says that your program is correct, then it is (modulo the bound), assuming everything else on the system (e.g. the Rust implementation) is also correct. Sometimes there are bit flips[2]. When fuzzing reports that it can't find anything, you also don't really know that your program is correct, but you have some warm and fuzzy feelings (sorry).

Under the hood. The blog post talks about this in just a bit of detail, and it's nothing that we haven't seen. Bolero uses `kani::any()` to generate a symbolic value, and `kani::assume()` to restrict the range of that value, if needed, e.g.

```
bolero::check!().with_generator(5..37).for_each( // snip... );
```

becomes

```
let value = kani::any();
kani::assume((min..max).contains(&value));
value
```

Kani (BMC) to Dafny (auto-active program verification)

Next, we are going to start talking about Dafny, which is a tool for automatically verifying annotations in program code. As I've alluded to before, Dafny makes it possible to mostly-automatically do what you have done manually in SE 212. Some people call it an "auto-active" system (versus interactive theorem provers).

We'll compare Dafny and Kani, using a different example[3].

Here's a Rust Rectangle implementation.

```
#[derive(Debug, Copy, Clone)]
struct Rectangle {
    width: u64,
    height: u64,
}

impl Rectangle {
    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }

    fn stretch(&self, factor: u64) -> Option<Self> {
        let w = self.width.checked_mul(factor)?;
        let h = self.height.checked_mul(factor)?;
        Some(Rectangle {
            width: w,
            height: h,
```

[2] https://mas.to/@gabrielesvelto/116171750653898304

[3] https://model-checking.github.io/kani-verifier-blog/2022/05/04/announcing-the-kani-rust-verifier-project.html

```
        })
    }
}
```

Pretty standard code. I can also provide the Dafny version:

```
class Rectangle {
    var width: int
    var height: int
}

predicate can_hold(self: Rectangle, other: Rectangle)
    reads self, other
{
    self.width > other.width ∧ self.height > other.height
}

method stretch(self: Rectangle, factor: nat) returns (rv : Rectangle) {
    rv := new Rectangle;
    rv.width := self.width * factor;
    rv.height := self.height * factor;
    return rv;
}
```

It would probably be more idiomatic object-oriented code to put stretch on the Rectangle, but it doesn't really matter for our purposes. In terms of proofs, we could state and maintain class invariants, which you should have seen in CS 247.

Going back to Rust, we can write a test case:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn stretched_rectangle_can_hold_original() {
        let original = Rectangle {
            width: 8,
            height: 7,
        };
        let factor = 2;
        let larger = original.stretch(factor);
        assert!(larger.unwrap().can_hold(&original));
    }
}
```

This is a test case which checks one specific input to can_hold. It succeeds, which is nice, but doesn't tell us that much.

We also know how to write Dafny test cases:

```
method {:test} TestStretchedRectangleCanHoldOriginal() {
    var original := new Rectangle;
    original.width := 8; original.height := 7;
    var factor := 2;
    var larger := stretch(original, factor);
    expect can_hold(larger, original);
}
```

We can replace the expect with an assert, but that won't verify, because there is no postcondition for method stretch. If we add a postcondition, then the proof for the test case goes through, but that's still not a general result. It's just about that particular rectangle.

There is a Rust crate (library) that does property-based testing, where it generates thousands of test cases. We can run both of these tests using cargo test.

```rust
#[cfg(test)]
mod proptests {
    use super::*;
    use proptest::prelude::*;
    use proptest::num::u64;

    proptest! {
        #[test]
        fn stretched_rectangle_can_hold_original(width in u64::ANY,
            height in u64::ANY,
            factor in u64::ANY) {
            let original = Rectangle {
                width: width,
                height: height,
            };
            if let Some(larger) = original.stretch(factor) {
                assert!(larger.can_hold(&original));
            }
        }
    }
}
```

This succeeds too. But we can go one step further and verify that the property that we've tested holds for any input, by running cargo kani --harness stretched_rectangle_can_hold_original.

```rust
#[cfg(kani)]
mod verification {
    use super::*;

    #[kani::proof]
    pub fn stretched_rectangle_can_hold_original() {
        let original = Rectangle {
            width: kani::any(),
            height: kani::any(),
        };
        let factor = kani::any();
        if let Some(larger) = original.stretch(factor) {
            assert!(larger.can_hold(&original));
        }
    }
}
```

This proof harness tells Kani to test the behaviour for *any* width, height, and factor. When we invoke Kani, we see that it runs a whole bunch of tests, looking mostly for safety violations (in unsafe Rust, of which there is none here). However, Kani also reports that the assertion fails. Why?

It turns out that we're missing some preconditions. Kani does support contracts like those we've

seen in Dafny, but we'll just encode them directly in the test harness.

```
kani::assume(0 != original.width);   //< explicit requirements
kani::assume(0 != original.height);  //<
kani::assume(1 < factor);            //<
```

With these assumes (effectively preconditions), Kani verifies the code.

Similarly, in Dafny:

```
method stretch(self:Rectangle, factor:nat) returns (rv : Rectangle)
    ensures rv.width = self.width * factor ∧ rv.height = self.height * factor
{
    rv := new Rectangle;
    rv.width := self.width * factor;
    rv.height := self.height * factor;
    return rv;
}

method stretched_rectangle_can_hold_original(original:Rectangle, factor:nat)
    requires original.width > 0 ∧ original.height > 0 ∧ factor > 1
{
    var larger := stretch(original, factor);
    assert can_hold(larger, original);
}
```

Dafny Application: Cedar authorization-policy language

I think some people are using Dafny "for real". Here's an example I managed to find, about the Cedar authorization-policy language[4] at Amazon. It looks like the formalization was later migrated from Dafny's automatic verification to another system, Lean, which supports interactive theorem proving (not as automatic, but more powerful), and there is a description of Cedar and Lean in [CDE+24].

Here is the blog post about Dafny and Cedar.

https://www.amazon.science/blog/how-we-built-cedar-with-automated-reasoning-and-differential-testing

What Cedar does. A Cedar user writes policies in a domain-specific language and then can execute queries on the policies, which may have response "allow" or "deny". Consider a hypothetical todo list manager TinyTodo. It may have the following policies, written in Cedar's language:

```
// policy 1
permit(principal, action, resource)
when {
  resource has owner && resource.owner == principal
};
```

From the blog post: "This policy states that any principal (a TinyTodo **User**) can perform any action on any resource (a TinyTodo **List**) as long as the resource's creator, defined by its **owner**

[4] https://www.cedarpolicy.com/en

attribute, matches the requesting principal." This above policy that enables access in some cases. Here's one below that forbids access.

```
// policy 3
forbid (
 principal in Team::"interns",
 action == Action::"CreateList",
 resource == Application::"TinyTodo"
);
```

What we have here is that interns can't create new lists for application TinyTodo.

Given a complete set of policies, then TinyTodo doesn't need to do the reasoning about authorization; it can just ask Cedar whether a given action is to be allowed or denied, and proceed accordingly.

Verification-guided development. The point of Dafny (and Lean) is to be able to prove things about software. We write some properties and some code and formally verify that the code respects the properties. It's better than testing, in that the property is guaranteed to be true of the code.

Maybe we don't actually want to deploy Dafny code in production. (Actually, it is possible to compile Dafny to other languages and deploy that). In this application, the Dafny code we're talking about is best thought of as a model of code written in some other language (in this case, Rust). It is still useful to verify the properties on the Dafny implementation, because we'll know that it is possible to write code respecting them. Plus, the Dafny code can serve as an oracle, which is useful, and we can look for divergences between the Dafny code and the Rust code.

But, really, we want to know that the Rust code satisfies the properties. Safe Rust is going to ensure that there is no undefined behaviour, so that's useful, but you do not get any guarantees that the behaviour respects the contracts.

You can put in assertions, but what do you do if they fail?

The approach here is to use *differential random testing* to make sure that the Rust implementation matches the Dafny model. This is like fuzzing: generate millions of inputs and check that Rust and Dafny do the same thing.

Properties of Cedar. There are two overarching properties that Cedar must satisfy when it responds to queries. Quoting the post again:

- *explicit permit*: permission is granted only by individual **permit** policies and is not gained by error or default;
- *forbid overrides permit*: any applicable **forbid** policy always denies access, even if there is a **permit** policy that allows it.

So, authorization has to be explicitly granted by some specific policy, and if there is any forbid policy that affects a request, then access is forbidden.

For instance, a request might be *principal*=**User::"Alice"**, *action*=**Action::"GetList"**, and *resource*=**List::"AliceList"**. Applying that to policy 1, we get expression **List::"AliceList" has owner && List::"AliceList".owner == User::"Alice"**. This is probably going to be true, in which case the request *satisfies* the policy. More generally, the authorization engine computes the request over all of its policies and returns an overall decision.

Some Bad Code. OK, let's look at some buggy Dafny code that tries to compute whether a request is authorized.

```
function method isAuthorized(): Response { // BUGGY VERSION
  var f := forbids();
  var p := permits();
  if f ≠ {} then
    Response(Deny, f)
  else
    Response(Allow, p)
}
```

Can you see why this code is wrong? Well, it does correctly capture forbidding a request if there is some reason to forbid it. But we also want a policy that says that there has to be an explicit permit for a request to be allowed, and this code doesn't respect such a policy. We can encode that policy as a predicate (which uses a lemma).

```
// A request is explicitly permitted when a permit policy is satisfied
predicate IsExplicitlyPermitted(request: Request, store: Store) {
  ∃ p •
    p in store.policies.policies.Keys ∧
    store.policies.policies[p].effect = Permit ∧
    Authorizer(request, store).satisfied(p)
}

lemma AllowedIfExplicitlyPermitted(request: Request, store: Store)
ensures // A request is allowed if it is explicitly permitted
        (Authorizer(request, store).isAuthorized().decision = Allow) ⟹
        IsExplicitlyPermitted(request, store)
{...}
```

The predicate is saying that there has to be a specific policy `p` which permits `request`. The lemma is something that needs to be shown true; here, it is saying that if the authorizer says yes, then the predicate `IsExplicitlyPermitted` has to be true. When Dafny verifies the function `isAuthorized`, it tries to check that the lemma holds. But it doesn't, and supposedly Dafny reports "A postcondition might not hold on this return path" pointing at the ensures clause.

```
// BUGGY VERSION
function method isAuthorized():
    Response {
var f := forbids();
var p := permits();
if f ≠ {} then
  Response(Deny, f)
else
  Response(Allow, p)
}
```

```
// FIXED VERSION
function method isAuthorized():
    Response {
  var f := forbids();
  var p := permits();
  if f = {} ∧ p ≠ {} then
    Response(Allow, p)
  else
    Response(Deny, f)
}
```

Dafny verifies the fixed version, and also the mirror property *forbid overrides permit*, which we don't show.

Dafny also can show other properties, including that the Cedar *validator* is *sound*: as they put it, "if the validator accepts a policy, evaluating the policy should never result in certain classes of error". Making this proof go through in Dafny enabled the developers to fix subtle bugs in the design, which would otherwise have carried over into the implementation.

The specification is useful for automated reasoning by Dafny, but also for manual reasoning by people. Even good Rust is still an implementation language with lots of implementation details. Reading the Dafny (or Lean) is much easier, since it has much less code (1/6th at the time of report).

Dafny and Rust. OK, so we can formally verify the Dafny. The thing about models is that I have always worried about making sure that the actual implementation conforms to the model. If you have no proof of correspondence, then, sure, the model can help, but bugs can sneak into the implementation as well. The solution here is *differential random testing*.

The Amazon people used techniques that we've talked about in this class to overcome this problem. They randomly generated millions of inputs (access requests, data, and policies) and check that Dafny (now Lean) and Rust produce the same output. If they disagree, there is a bug.

The inputs aren't completely randomly generated. They used the same techniques that we've talked about in this class—they use input generators to create "policies, data and requests that are consistent with one another". They report 6 hours nightly of DRT and 100 million total tests. They found errors in their libraries (IP address operations and parsing, as well as the policy parser).

References

[CDE+24] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proc. ACM Program. Lang.*, 8(OOPSLA1), April 2024.